

Project02

컴퓨터소프트웨어학부 2022095287 김준서

Setting

- 실행 환경: Ubuntu-22.04

Design

- Array-based Circular Queue 자료구조 사용
 - 큐에 대한 락은 `ptable.lock` 과 critical section이 겹치므로 해당 락을 사용
- 기존의 `proc` 구조체에 `priority`, `queue_level`, `run_ticks` 속성 추가
- 글로벌 틱인 `gticks` 를 새로 정의. 락은 기존 `ticks` 와 동일한 `tickslock` 을 사용
- 인터럽트 핸들러
 - 매 tick마다 `yield` 하던 로직 삭제
 - 프로세스의 `run_ticks` 와 큐의 `time_quantum` 을 비교하여 time runout 판단 후 `yield`
 - `gticks` 이 100의 배수가 될 때마다 priority boosting. `gticks` 이 int 범위를 벗어날 때까지는 충분히 오랜 시간이 필요하므로 초기화하지 않아도 된다고 판단.

Implementation

System Call

- `sysproc.c`: 각각의 시스템 콜에 대해 `proc.c` 의 함수를 적절하게 호출하는 wrapper 함수 추가
- 시스템 콜이 호출될 수 있도록 `user.h`, `usys.S`, `syscall.c`, `syscall.h`, `Makefile` 수정

Queue

- array based circular queue를 구현한 `queue` 자료구조 사용
 - 최대 프로세스 개수가 `NPROC` 로 제한되어 있음
 - Linked List에 비해 구현이 간단함
- 큐에는 `RUNNABLE` 또는 `RUNNING` 중인 프로세스만 저장
 - `p->state` 가 `RUNNABLE` 이 될 때(`fork`, `userinit` 등): `queue_push` 를 호출
 - `p->state` 가 `SLEEPING` 또는 `ZOMBIE` 가 될 때: 직후에 `sched` 를 호출하므로, `sched` 에서 `myproc()->state` 기반으로 `queue_delete` 를 호출
- 락은 `ptable.lock` 과 공유
 - `ptable` 로의 접근 없이 큐의 정보만 바꿀 일이 없음
- MLFQ와 MoQ는 각각 `queue` 타입 전역 변수 `L[4]`, `MoQ` 로 정의
- 구조체 정보

```

struct queue {
    struct proc *arr[NPROC];
    int front, rear;
    int size;
    int time_quantum;    // allocated time quantum
    char name[10];       // queue name (debugging)
    int level;           // queue level
};

```

- APIs: 모든 함수는 호출 전에 `ptable.lock` 을 획득했음을 가정하고 구현됨.

API	설명	예외	비고
<code>int queue_isempty(struct queue *q)</code>	비어있는지 여부 반환		
<code>int queue_isfull(struct queue *q)</code>	꽉 차있는지 여부 반환		
<code>int queue_push(struct queue *q, struct proc *p)</code>	맨 뒤에 프로세스 추가	큐가 꽉 차있으면 -1, 성공적이면 0 반환	
<code>int queue_delete(struct queue *q, struct proc *p)</code>	특정 프로세스 제거	해당 프로세스가 없으면 -1, 성공적이면 1 반환	
<code>struct proc* queue_front(struct queue *q)</code>	맨 앞 원소 반환	큐가 비어있으면 0 반환	
<code>struct proc* queue_top(struct queue *q)</code>	최우선순위 프로세스 반환	큐가 비어있으면 0 반환	<code>NPROC</code> 이 작아 선형 탐색으로 구현
<code>void queue_init(struct queue *q, int qlen, int time_quantum, char* name)</code>	큐 초기화		
<code>void queue_move(struct queue *from, struct queue *to, struct proc *p)</code>	프로세스를 <code>from</code> 큐에서 <code>to</code> 큐로 이동	<code>p</code> 가 <code>from</code> 에 들어있지 않으면 <code>panic</code>	<code>from=to</code> 인 경우 가장 앞의 원소를 가장 뒤로 보내는 연산과 동치
<code>void queue_print(struct queue *q)</code>	프로세스들의 정보 출력		

proc.h

- 기존 `proc` 구조체에 다음 속성들을 추가
 - `int priority`: 0과 10 사이의 우선순위
 - `int qlen`: 프로세스가 속한 큐 레벨. `SLEEPING` 상태일 경우, 가장 마지막에 속해있던 (되돌아가야 할) 큐 레벨
 - `int run_ticks`: 프로세스가 실행한 tick 수

proc.c

- 추가한 것
 - 전역 변수: `queue` 타입 `L[4]` 와 `MoQ`
 - 전역 변수: `int mflag` 와 `spinlock mflaglock` (`mflag`가 1이면 monopolized)
 - [`priorityboost` 함수]
 - `RUNNABLE`, `RUNNING`, `SLEEPING` 상태인 프로세스들의 큐 레벨을 0으로 설정
 - `RUNNABLE` 또는 `RUNNING` 상태라면 `queue_move` 로 속한 큐 또한 변경.
 - [`istimerunout` 함수]: `p->run_ticks` 와 `q->time_quantum` 을 비교하여, timerunout 여부 반환
 - [`ismonopolized` 함수]: `mflaglock` 을 고려하여 `mflag` 반환
 - 명세에 있는 시스템 콜
 - 한 함수에서 2개 이상의 예외가 발생하는 경우, 구현의 편의를 위해 가장 일찍 발생한 예외를 반환
 - [`getlev` 함수]: `myproc()->qlen` 반환
 - [`setpriority` 함수]: `ptable` 에서 알맞은 프로세스를 찾아 `p->priority` 변경
 - [`setmonopoly` 함수]: `password` 검사 후, `ptable` 에서 알맞은 프로세스를 찾아 `queue_move` 로 속한 큐 변경
 - [`monopolize` 함수]: `mflaglock` 을 고려하여 `mflag` 를 1로 변경
 - [`unmonopolize` 함수]: `mflaglock` 을 고려하여 `mflag` 를 0으로 변경. `tickslock` 을 고려하여 `gticks` 를 0으로 변경
- 변경한 것
 - [`userinit`, `fork`, `wakeup1`, `kill` 함수]: `p->state = RUNNABLE`; 직후에 해당 프로세스를 큐에 추가 (`queue_push`)
 - [`sched` 함수]
 - `myproc()->run_ticks` 를 0으로 초기화
 - `myproc()->state` 가 `SLEEPING` 또는 `ZOMBIE` 일 경우, 해당 프로세스를 큐에서 삭제 (`queue_delete`)
 - `myproc()->state` 가 `RUNNABLE` 일 경우, 큐 레벨에 따라 프로세스를 적절한 큐로 이동 (원래 큐일 수도 있음, `queue_move`)
 - [`scheduler` 함수]
 - `mflaglock` 을 고려하여 `mflag` 값 확인
 - `mflag=1`
 - `MoQ` 큐가 비어있지 않다면 가장 앞의 원소로 스케줄링
 - 비어있다면 `unmonopolize` 시스템콜 호출
 - `mflag=0`

- L0, L1, L2 큐를 순서대로 확인하며 비어있지 않다면 RR로 스케줄링
- L3를 우선순위 스케줄링

trap.c

- 매 tick마다 `yield` 하던 로직 삭제
- 프로세스의 `run_ticks`와 큐의 `time_quantum`을 비교하여 time runout 판단 후 `yield`
- `gticks`이 100의 배수가 될 때마다 priority boosting
- `gticks`이 int 범위를 벗어날 때까지는 충분히 오랜 시간이 필요하므로 초기화하지 않아도 된다고 판단

Result

- 빌드 커맨드: `./buildev6.sh; ./bootev6.sh`
- xv6를 실행한 후, `m1fq_test` 실행

<pre>[Test 1] default Process 5 L0: 14490 L1: 30129 L2: 0 L3: 55381 MoQ: 0 Process 7 L0: 17268 L1: 27994 L2: 0 L3: 54738 MoQ: 0 Process 4 L0: 19068 L1: 0 L2: 50831 L3: 30101 MoQ: 0 Process 6 L0: 19431 L1: 0 L2: 51158 L3: 29411 MoQ: 0 Process 9 L0: 19978 L1: 37905 L2: 0 L3: 42117 MoQ: 0 Process 11 L0: 19739 L1: 39719 L2: 0 L3: 40542 MoQ: 0 Process 8 L0: 18828 L1: 0 L2: 56771 L3: 24401 MoQ: 0 Process 10 L0: 20021 L1: 0 L2: 57530 L3: 22449 MoQ: 0 [Test 1] finished</pre>	<pre>[Test 2] priorities Process 19 L0: 15181 L1: 19988 L2: 0 L3: 64831 MoQ: 0 Process 18 L0: 15011 L1: 0 L2: 29799 L3: 55190 MoQ: 0 Process 17 L0: 14742 L1: 26603 L2: 0 L3: 58655 MoQ: 0 Process 16 L0: 19858 L1: 0 L2: 44053 L3: 36089 MoQ: 0 Process 15 L0: 19240 L1: 38509 L2: 0 L3: 42251 MoQ: 0 Process 14 L0: 19263 L1: 0 L2: 44792 L3: 35945 MoQ: 0 Process 13 L0: 21276 L1: 39396 L2: 0 L3: 39328 MoQ: 0 Process 12 L0: 16440 L1: 0 L2: 23047 L3: 60513 MoQ: 0 [Test 2] finished</pre>	<pre>[Test 3] sleep Process 20 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 21 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 22 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 23 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 24 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 25 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 26 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 Process 27 L0: 500 L1: 0 L2: 0 L3: 0 MoQ: 0 [Test 3] finished</pre>	<pre>[Test 4] MoQ Number of processes in MoQ: 1 Number of processes in MoQ: 2 Number of processes in MoQ: 3 Number of processes in MoQ: 4 Process 29 L0: 0 L1: 0 L2: 0 L3: 0 MoQ: 100000 Process 31 L0: 0 L1: 0 L2: 0 L3: 0 MoQ: 100000 Process 33 L0: 0 L1: 0 L2: 0 L3: 0 MoQ: 100000 Process 35 L0: 0 L1: 0 L2: 0 L3: 0 MoQ: 100000 Process 28 L0: 9942 L1: 0 L2: 28493 L3: 61565 MoQ: 0 Process 30 L0: 10084 L1: 0 L2: 30450 L3: 59466 MoQ: 0 Process 32 L0: 10065 L1: 0 L2: 29057 L3: 9897 L1: 0 L2: 30909 L3: 59194 MoQ: 0 L3: 60878 MoQ: 0 [Test 4] finished</pre>
---	--	---	--

Trouble Shooting

1. RUNNABLE한 프로세스가 없는 경우 어떻게 할 것인가?
 - 생길 때까지 busy waiting
 - 기본 xv6 스케줄러 역시 busy waiting으로 동작한다
2. 어느 시점에 큐 자료구조에 프로세스를 추가 / 삭제할 것인가?
 - `p->state`를 `RUNNABLE`로 바꾸는 시점
 - `sched`에서 스케줄러로 컨텍스트 스위칭이 일어나기 전 시점
3. 추가한 전역 변수들의 lock을 어떻게 관리할 것인가?

- 큐: `ptable` 과 함께 접근하므로, `ptable.lock` 이 `queue lock` 도 암시한다고 간주
- `mflag: mflaglock` 추가
- `gticks`: 기존의 `ticks` 와 `tickslock` 공유

4. 큐에 저장할 프로세스들의 상태는 무엇인가?

- RUNNING / RUNNABLE 프로세스만 큐에 저장하기로 결정
- `unmonopolize`, `priority boosting` 같이 SLEEPING도 고려해야 한다면, `ptable` 을 참조

5. 어떤 프로세스가 RUNNING -> SLEEP -> RUNNABLE 상태가 되었을 때, 기존 큐 레벨에 넣어줄 것인가 or L0에 넣어줄 것인가?

- 기존 큐 레벨에 넣어주기로 결정
- `p->qlev` 는 `p` 가 SLEEPING인 경우, 되돌아가야 하는 큐 레벨을 저장하게 됨